Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — The ticket, and the agent that fills it
- Hint 2 — The plan, and every failure

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The ticket, and the agent that fills it

Write the ticket as a Pydantic model in `src/schemas.py` — the 4 fields from the README's Real Example. Two details matter:

- `customer_name: str | None` — `None` must be a valid answer, or the model will invent a name.
- `Literal[...]` for category and urgency, so only your exact words can come back.

Then `src/agents/triage.py` (plus an empty `agents/__init__.py`). The agent sends 2 messages — its system prompt and the complaint — with **no** chat history. It uses Doc04's `client.beta.chat.completions.parse(model=..., messages=..., response_format=SupportTicket)` and reads `completion.choices[0].message.parsed`.

```
def triage_agent(client: OpenAI, message: str) -> SupportTicket | None:
    ...
```

It returns either a ticket or `None` — never anything half-valid:

- Wrap the `.parse(...)` call in `try` / `except ValidationError` → log a warning, return `None`.
- Check `reply.refusal` **before** `reply.parsed` (Doc04) → log it, return `None`.
- Otherwise return `reply.parsed`.

Put the system prompt in `prompts/triage.txt` and load it with `load_prompt("triage")`. Make it strict: "leave customer_name null unless the customer states their name — never guess one". The `None` in the schema only *allows* an empty name; the prompt is what *asks* for it.

Get the logger inside the function (`logger = get_logger(__name__)`), not at the top of the file. Files are imported before `load_config()` runs, so a top-level logger would miss `LOG_LEVEL` from `.env`.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and every failure

In the loop, before the chat branch:

```
if message.startswith("/extract "):
    text = message.removeprefix("/extract ")
    ticket = triage_agent(client, text)
    if ticket is None: print a "couldn't turn that into a ticket" line
    else: print_ticket(ticket)
else:
    (Step 2's chat: trim, then stream)
```

`print_ticket()` goes in `services/desk_service.py` (with an empty `services/__init__.py`), so Step 4 can add the rest of the flow to the same file. It prints the 4 fields on their own lines, with `(not given)` when the name is `None`.

Put the whole `if/else` inside a `try:`. The `except` blocks go right after it (below).

One `except` per failure, each with the reaction from Doc04's error table:

Catch	Why it happens	What to do
`openai.AuthenticationError`	key revoked mid-session	log error, print, `sys.exit(1)`
`openai.RateLimitError`	too many requests	log a warning, ask the user to wait
`openai.BadRequestError`	message too long	if `history[-1]["role"] == "user"`: `history.pop()`, then tell the user

Why `history.pop()`: `stream_message()` appended the user turn *before* the call failed. Left there, every later turn would resend it and fail again.

Then the files that make the step checkable:

- `data/sample/messages.json` — `{"triage": [ ...the 3 inputs... ]}`.
- `scripts/triage_samples.py` — read the file and `json.loads()` its text (Doc02), run `triage_agent()` on each message, print each ticket.

- `tests/unit/test_schemas.py` — a ticket with `customer_name=None` is accepted; `urgency="urgent"` raises `ValidationError`. Each case prints `OK` or `WRONG`.
- `tests/integration/test_live_agents.py` — two new test functions: every sample becomes a ticket (case 4), and input 2 gets `customer_name=None` (case 5).

Full solution: Show me the solution